CMPE172 Milestone 1 Report — CareConnect

1. Project proposal

CareConnect is a clinic appointment-booking system. It helps patients browse healthcare services and available appointment slots, then eventually book and manage appointments with healthcare providers. Providers can create availability and view appointments assigned to them.

The system has two primary roles:

- **Customer/patient:** browses services and available slots, books appointments, views appointments, and cancels their own appointments.
- **Provider:** a doctor, nurse, therapist, or other healthcare professional who offers services, creates availability, and views appointments booked with them.

The twelve planned features are:

1. Patient login.
2. Browse, filter, and paginate available slots by provider, service, and date using SQL `LIMIT` and `OFFSET`.
3. Book an available slot for a chosen service.
4. View upcoming and historical appointments.
5. Cancel an appointment as the appointment owner.
6. Provider login.
7. Provider creates or removes service-specific availability slots.
8. Provider views appointments booked with them.
9. A mock external notification service sends confirmation messages after booking or cancellation.
10. Logging, a health endpoint, and at least one metric.
11. A conversational booking agent and an autonomous availability agent.
12. Grounded Q&A over a small clinic knowledge base using retrieval-augmented generation.

Milestone 1 implements only the database-backed skeleton and the two read endpoints `GET /` and `GET /slots`. Features 1–8 are planned for Milestone 2, features 9–10 for Milestone 3, and features 11–12 for Milestone 4.

2. Technology and architecture

The project is implemented in Java using Spring Boot. It uses SQL through JDBC to communicate with an H2 relational database. No ORM framework, JPA, Hibernate, or Spring Data JPA is used.

The request flow is:

Client → Controller → Service → Repository/JDBC → SQL database

The Controller receives HTTP requests. The Service coordinates application behavior. The Repository executes handwritten SQL through `JdbcTemplate`, maps result rows into DTOs, and returns them to the Service. The Controller returns the DTOs as JSON.

A future booking or cancellation operation will also call a mock external notification service. This boundary is shown in the architecture diagram, but the mock notification feature is planned for Milestone 3.

This project realizes classic enterprise standards on a modern stack: Spring Boot provides a modern equivalent of J2EE-style enterprise application development, while REST provides a modern approach to distributed-object communication historically associated with CORBA.

3. Database entities

The five required entities are `users`, `providers`, `services`, `availability_slots`, and `appointments`.

users

Stores login and role information for patients, providers, and administrators. The primary key is `user_id`; `email` is unique.

providers

Stores provider-specific information. `user_id` is a foreign key to `users.user_id` and is unique, so one user has at most one provider profile.

services

Stores clinic services such as general consultation and physical therapy. A service can be offered in many availability slots.

availability_slots

Stores provider-specific service times. Each slot references exactly one provider and one service. `end_time` must be later than `start_time`.

appointments

Stores bookings made by patients. Each appointment references one patient and one slot. The `UNIQUE(slot_id)` constraint prevents two appointment rows from using the same slot and therefore prevents double-booking at the database level.

4. Relational schema

USERS(
  user_id PK,
  name,
  email UNIQUE,
  password_hash,
  role

PROVIDERS(
provider_id PK,
user_id FK → USERS.user_id UNIQUE,
specialization,
bio,

```
license_number
)

SERVICES(
service_id PK,
service_name UNIQUE,
description,
duration_minutes,
price
)

AVAILABILITY_SLOTS(
slot_id PK,
provider_id FK → PROVIDERS.provider_id,
service_id FK → SERVICES.service_id,
start_time,
end_time,
status
)

APPOINTMENTS(
appointment_id PK,
slot_id FK → AVAILABILITY_SLOTS.slot_id UNIQUE,
customer_id FK → USERS.user_id,
status,
booked_at
)
```

Cardinality is as follows: one provider has many slots; one service has many slots; one patient has many appointments; and one slot has zero or one appointment. An appointment is existence-dependent on a patient and a slot because both foreign keys are mandatory. It has its own primary key, so it is modeled as a regular entity with mandatory dependencies rather than a strict weak entity.

5. Page Controller and Front Controller

A Page Controller has a controller or handler focused on a particular page or user action. A server-rendered Thymeleaf application often resembles this pattern because separate handlers render pages such as Home, Available Slots, Booking, and Confirmation.

A Front Controller provides one central entry point for requests. In Spring MVC, `DispatcherServlet` receives incoming HTTP requests and dispatches them to the correct controller method. CareConnect exposes REST endpoints, so the request enters through Spring's front-controller infrastructure and is then routed to `CareConnectController`.

6. Milestone 1 implementation

The project contains `schema.sql` and `seed.sql`. Spring Boot loads both files during startup. The application includes a `CareConnectController`, `CareConnectService`, `SlotRepository`, `HomeDto`, and `AvailableSlotDto`.

For `GET /`, the Controller calls the Service, which calls the Repository. The Repository executes a SQL count query and returns a `HomeDto` containing the application name, message, and number of available slots.

For `GET /slots`, the Repository joins availability slots, providers, users, and services. It maps the result rows into `AvailableSlotDto` objects. The Controller returns those DTOs as JSON.

7. Running the project

From the `careconnect` directory:

```
mvn spring-boot:run
```

Then open:

http://localhost:8080/
http://localhost:8080/slots

8. Limitations and future work

Authentication, booking transactions, cancellation, filters, pagination, notifications, logging, health metrics, and AI features are intentionally outside the Milestone 1 implementation scope. They are represented in the proposal and database design for future milestones.